

Python Advanced Problem Set (Set 2)

Advanced Logic, Data Wrangling, Algorithmic Functions & OOP Architectures

Course Component

Problem Set 2

Total Questions

20 Problems

Target Focus

NumPy & Pandas Transition

Objective: This problem set challenges students to handle complex data manipulation logic, generator pipelines, advanced OOP design patterns, and custom data processing—all built from pure Python prior to starting NumPy and Pandas.

MODULE 1: ADVANCED CONTROL FLOW & ALGORITHMIC LOGIC

Level 1 – Intermediate

Q21. Run-Length Encoding & Compression

Write a function `compress_string(s)` that performs basic run-length encoding on an alphanumeric string (e.g., "aabccccaaa" becomes "a2b1c5a3"). If the compressed string is not smaller than the original, return the original string.

Q22. Longest Consecutive Sequence Finder

Write a function `longest_consecutive(nums)` that takes an unsorted list of integers and finds the length of the longest consecutive elements sequence in $O(n)$ time using sets.

Q23. Sliding Window Moving Average

Write a function `moving_average(data, window_size)` that computes the simple moving average over a numerical list using a sliding window approach. Return a list of averages rounded to 2 decimal places.

Q24. Pascal's Triangle Generator

Write a function `generate_pascals_triangle(num_rows)` that outputs the first N rows of Pascal's Triangle as a nested list of integers.

Q25. Valid Anagram & Grouping Engine

Write a function `group_anagrams(words)` that takes a list of strings and groups anagrams together into a dictionary where sorted string representations act as unique keys.

Q26. Tabular CSV-like Dict Alignment

Write a function `align_records(records)` that takes an inconsistent list of dict records (where some keys may be missing across entries) and standardizes them into a uniform dictionary with default values for missing keys.

Q27. Matrix Multiplication (Pure Python)

Write a function `matrix_multiply(A, B)` that performs matrix multiplication between two 2D lists. Validate that column dimensions of `A` equal row dimensions of `B`; otherwise raise a custom `DimensionMismatchError`.

Q28. Nested JSON Key Extraction

Write a recursive function `extract_nested_keys(d, target_key)` that searches through arbitrary nested dictionary structures (like JSON responses) and collects all instances matching a specific key.

Q29. Custom Pivot Table Simulation

Write a function `pivot_summary(data, group_by_col, value_col)` that takes a list of dictionary rows, groups values by `group_by_col`, and aggregates the sum, average, and count for `value_col`.

Q30. Priority Task Queue (Min-Heap / Dict Logic)

Implement a priority task dispatcher function `process_tasks(tasks)` where each task is a dict with `{"name": str, "priority": int}`. Process and return task names in order from highest priority (lowest integer value) to lowest.

Q31. Execution Time Tracker Decorator

Create a decorator function `@time_execution` that logs the elapsed runtime in milliseconds for any function it decorates, printing execution metadata automatically upon function completion.

Q32. Lazy Evaluation File Batcher (Generator)

Write a generator function `chunk_file_reader(filepath, chunk_size=100)` that reads a large file lazily line-by-line and yields list chunks of size `chunk_size` to conserve memory.

Q33. Data Validation Decorator

Create a decorator `@enforce_types(type_schema)` that checks arguments passed to a function against a schema dictionary (e.g. `{"val": float}`) and raises a `TypeError` if invalid data types are supplied.

Q34. Custom Memoization Decorator

Implement a caching decorator `@memoize` that stores previously computed function results in a dictionary cache to speed up expensive recursive algorithms like computing Fibonacci numbers.

MODULE 4: ADVANCED OOP & DATA ANALYSIS PREP

Level 3 — Advanced

Q35. Custom Array / Matrix Container Class

Design a class `Array2D` that wraps a flat list and dimensions `(rows, cols)`:

- Overload `__getitem__` and `__setitem__` to allow tuple indexing like `arr[r, c]`.
- Include a `reshape(new_rows, new_cols)` method that returns a re-indexed `Array2D` object.

Q36. Data Pipeline Filter Chain

Build a modular class structure for a data pipeline:

- `BasePipelineStep`: Abstract class with a method `process(data)`.
- Implement concrete steps: `FillMissingStep(default_val)`, `ScaleStep(factor)`, and `FilterOutliersStep(min_val, max_val)`.
- `Pipeline`: Class that chains steps and runs `execute(data)` sequentially.

Q37. Bank Transaction Ledger with Properties

Implement a class `AccountLedger` using Python `@property` syntax:

- Maintain a private transaction history log `__transactions`.
- Read-only properties: `balance` (computed dynamically) and `transaction_count`.
- Prevent direct mutation of ledger entries from outside the class.

Q38. Polymorphic File Exporter System

Build a data export module for dictionary datasets:

- Abstract class `DataExporter` with abstract method `export(data, filepath)`.
- Subclasses: `CSVExporter` (formats as comma-separated values) and `JSONExporter` (formats as indented JSON string).
- Class Factory: `ExporterFactory.get_exporter(format_type)` that returns appropriate instances.

Q39. Custom Mini-DataFrame Class (Pre-Pandas Master Task)

Build a `MiniDataFrame` class storing tabular data as a dictionary of column lists (e.g. `{"age": [25, 30], "salary": [50000, 60000]}`):

- **Methods:**
 - `info()`: Displays column names, non-null counts, and data types.
 - `describe()`: Computes count, mean, min, and max for each numerical column.
 - `filter_by(col_name, predicate_func)`: Returns a new filtered `MiniDataFrame` instance.

Q40. GroupBy & Aggregate Engine

Add advanced grouping capability to the `MiniDataFrame` class:

- Implement a method `groupby(col_name)` that returns a custom `GroupedData` object.
- Implement aggregation methods on `GroupedData` such as `sum()` and `mean()` that calculate group-level summary metrics.

Mastery Milestone: Completing all 40 questions (Sets 1 & 2) ensures students possess a complete mastery of Python core structures, advanced logic, file workflows, and OOP paradigms necessary to master NumPy and Pandas.